recodai-luc-scientific-image-forgery-detection

2nd Place Solution

Rank	2
Team	shiba-inu
Author	shiba-inu
Topic ID	694397
Version	Original

Recod.ai/LUC - Scientific Image Forgery Detection 2nd Place Solution

First of all, thank you to the organizers for hosting such a wonderful competition. It was a highly meaningful theme—protecting the integrity of scientific research—and I learned a great deal from it.

Result: **Public 3rd → Private 2nd**

Solution Overview

Rather than a deep learning-based approach, my solution is built around **classical feature matching (SIFT + RANSAC)**, combined with YOLO-based preprocessing and custom post-processing in a 3-stage pipeline.

```
Stage 1: Valid Region Detection (Panel/Text detection with YOLOv8)
    ↓
Stage 2: Feature Matching (SIFT + G2NN + RANSAC)
    ↓
Stage 3: Post-processing (Cluster merging + High-precision mask generation)
```

Stage 1: Valid Region Detection

In the subsequent feature matching stage, **graph regions** produce a large number of false positives, and **text regions** cause erroneous matching on caption parts (e.g., matching on the "mm" portion of "10 mm"). These lead to significant accuracy degradation, making their exclusion essential.

I trained a **Panel detection model** and a **Text detection model** using YOLOv8-m to address this problem.

Panel Detection

To exclude irrelevant regions such as graphs, only relevant areas (image regions) are detected. Simultaneously, YOLO classifies detections into 3 classes: **corn images**, **Western blot protein images**, and **other biological images**. The reason for separating classes is that the optimal RANSAC inlier threshold differs by image type (described below).

Since no suitable training data existed, I addressed this by **automatically generating synthetic data**.

- Used a subset of 2,000 images from external data (BioFors dataset) due to time constraints
- Classified and extracted cell/plant images from the competition's authentic images
- Auto-generated synthetic graphs (scatter plots, line charts, heatmaps, violin plots)
- Randomly arranged the above in various layout patterns (1×1, 2×2, 3×1, etc.) to create training data
- Ensembled 5 YOLOv8-m models trained with different seeds using **Weighted Boxes Fusion (WBF)**

Text Detection

Existing OCR solutions (EasyOCR, etc.) were insufficiently accurate—for example, misrecognizing round cells as the letter "O". Since text in scientific paper images has minimal distortion, I determined that OCR-specific heads were unnecessary and **trained YOLOv8-m as an object detector**.

- Manually corrected low-threshold EasyOCR detection results to create annotations (~800 images)
- Added synthetic data (~2,000 images) with randomly placed text on text-free images
- Applied diversity in background color, text color, font, rotation, and size for augmentation
- At inference, performed TTA with two image sizes (640, 1280) and merged both results

---

Stage 2: Feature Matching

Aggressive SIFT Keypoint Extraction

Biological images have very little texture, and default settings fail to extract sufficient keypoints.

- Used `contrast_threshold=0.001` (approximately 1/40 of the default 0.04) to extract a large number of keypoints
- For small images (<1024px), upscaled 4× before feature extraction, then mapped coordinates back to original scale

G2NN Matching

The absolute L2 distance values vary greatly across images, making fixed thresholds impractical.

Adopted **G2NN **: sort matching distances and accept only matches up to the breakpoint where $T_{n+1}/T_n > \alpha$ ($\alpha=0.7$). This achieves threshold-independent matching.

Speedup: KDTree

Due to the enormous number of keypoints, used FLANN KDTree approximate nearest neighbor search to reduce complexity from $O(N^2)$ to $O(N \log N)$.

RANSAC + Geometric Filtering

Two sampling strategies were employed:

- **Local sampling**: Execute RANSAC within high-density match regions + YOLO-detected bboxes. Analyze the H matrix to filter out unrealistic transformations (scale >4×, shear deformation, etc.)
- **Global sampling**: Execute RANSAC once on all matches, then apply HDBSCAN clustering to remove false matches

Additionally, RANSAC inlier thresholds were adjusted per image type based on the 3 classes detected in Stage 1:

- **Corn images**: Fine black-and-white line patterns cause frequent false matches, so the inlier threshold was set **high** for strict filtering
- **Protein images (Western blot)**: Very few features make matching inherently difficult, so the inlier threshold was set **low** to enable detection even with few matches
- **Other biological images**: An intermediate threshold was used

---

Stage 3: Post-processing

After feature matching, I discovered that **careful clustering is essential**. Without it, the F1 score (the competition metric) drops significantly. To achieve high-precision clustering, I implemented the following processing.

Two-stage Cluster Merging

1. **Stage 1 (within same pair)**: Merge nearby clusters with similar H matrices (normalized difference of rotation angle, scale, and translation < 0.02) using Union-Find
2. **Stage 2 (across different pairs)**: Merge as the same instance when IoMin (Intersection / min(area1, area2)) between source/dest bboxes ≥ 0.3

High-precision Mask Generation

1. Recompute H matrix from all corresponding points of merged clusters
2. Generate source/dest masks via convex hull + dilation
3. Refine masks precisely using H matrix transformation error (within mean + 2σ)
4. Apply bidirectional refinement using inverse transformation

---

What Didn't Work

- **Other feature descriptors** (SURF, ORB, AKAZE, SuperPoint): Could not extract sufficient keypoints in the biological image domain, resulting in poor accuracy
- **Deep learning-based approaches**: Tried CMFD models combining DINOv2/SegFormer backbones with self-correlation modules, but accuracy was insufficient

---

Summary

Component	Details
Panel Detection	YOLOv8-m × 5 (WBF ensemble), trained on synthetic data

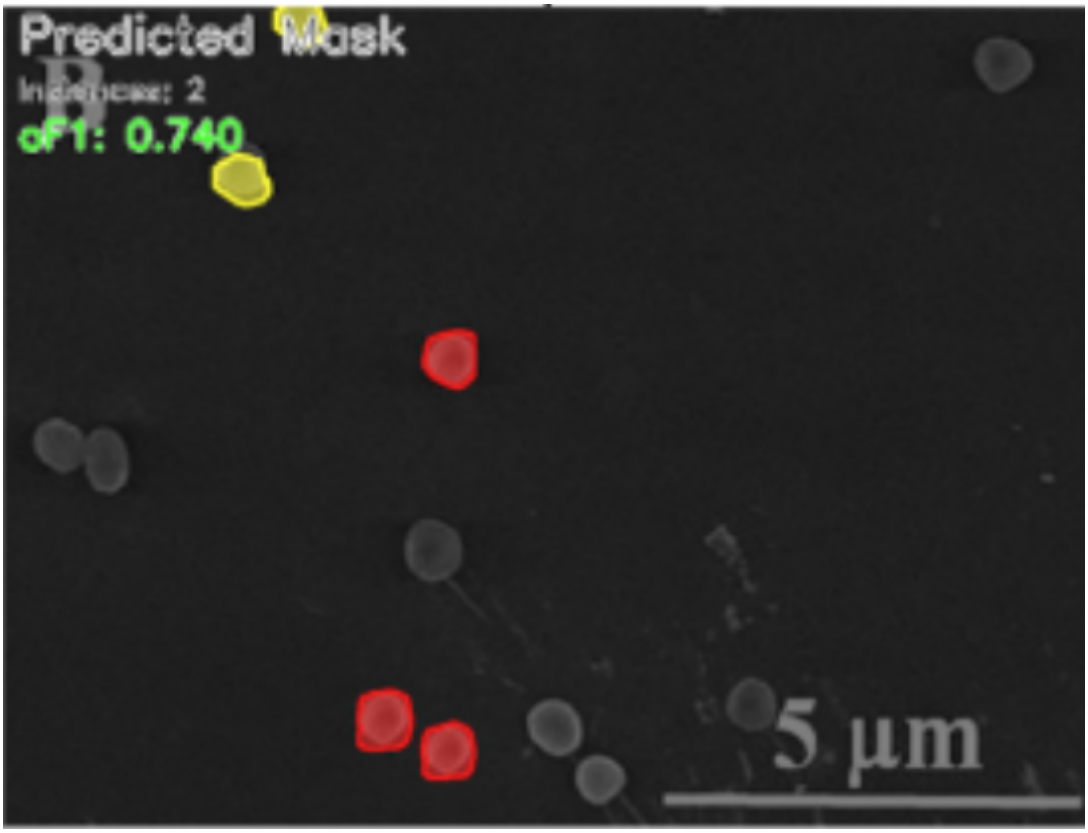
Component	Details
Text Detection	YOLOv8-m, manual annotation + synthetic data
Features	SIFT (contrast_threshold=0.001)
Matching	G2NN (α=0.7) + FLANN KDTree
Geometric Estimation	Affine RANSAC + H matrix filtering
Clustering	HDBSCAN + 2-stage merging (H similarity → IoMin)
Mask Refinement	Convex hull + H transformation error-based refinement

Code is available here https://github.com/oshima-yoppi/luc_pub.

Supplemental Q&A

> **CoreyJamesLevinson · 2026-04-28T01:39:51.067000**
>
> Altering SIFT/KDTree/RANSAC parameters by the panel type is a good idea. I also found that SIFT performs poorly on Western blots since there are not that many keypoints. It would be better to have a solution specifically designed for Western Blots.
>
> If I understand correctly, you did not look at any solution which dealt with duplicates inside of the same panel image. I tried forcing it by splitting up every detected panel into halves/thirds/quadrants/sixths. However, it could not complete within the 4 hour runtime. This is especially important for the Western Blot where lanes may be copied over within the same image.
>
> Big congrats on 2nd place! I will never look at shiba's the same way again:)

> **shiba-inu · 2026-04-28T15:30:06.473000**
>
> > It would be better to have a solution specifically designed for Western Blots.
>
> Definitely yes. I struggled a lot with Western blot–type images. It was very difficult to make the method work reliably, and after trying various approaches, I ended up using spatial feature extraction methods such as keypoint-based techniques.
>
> > you did not look at any solution which dealt with duplicates inside of the same panel image
>
> Sorry if I misunderstood your point. In my approach, I am actually able to detect duplicated regions within the same panel. As shown in the example image (which was generated by my algorithm), duplicated regions inside a panel can be successfully identified—even in cases where the entire image is treated as a single panel.


Predicted Mask
oF1: 0.740
5 µm